



Shri Shamrao Patil (Yadravkar) Educational & Charitable Trust's

Sharad Institute of Technology

College of Engineering, Yadrav (Ichalkaranji)

An Autonomous Institute

Department of Electronics and Computer Engineering

A

Mini Project Report

On

**FINGERPRINT BASED BIOMETRIC ATTENDANCE
SYSTEM WITH GOOGLE SHEET**

SUBMITTED BY

Roll No.

Student Name

26

Mr. ADITYA PRAKASH HALADE

27

Mr. RUSHIKESH SAMADHAN ALDAR

31

Mr. DARSHAN SUDHAKAR BUCHADE

35

Mr. ABHIJIT HARISHCHANDRA DALAVI

Under the Guidance of

Mr.S.R.Hawal

Department of Electronics and Computer Engineering,

Sharad Institute of Technology College of Engineering, Yadrav-Ichalkaranji

2024-25



Shri Shamrao Patil (Yadravkar) Educational & Charitable Trust's
Sharad Institute of Technology College of Engineering
Yadrav (Ichalkaranji) - 416121

An Autonomous Institute

(Approved by AICTE, New Delhi, Recognized by Government of Maharashtra & Affiliated to Dr. Babasaheb Ambedkar Technological University)

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

This is to certify that,

Student Name	Roll No.
MR. ADITYA PRAKASH HALADE	(26)
MR. RUSHIKESH SAMADHAN ALDAR	(27)
MR. DARSHAN SUDHAKAR BUCHADE	(31)
MR. ABHIJIT HARISHCHANDRA DALAVI	(35)

Studying in SY Electronics and Computer Engineering has successfully completed the Mini Project Synopsis **“FINGERPRINT BASED BIOMETRIC ATTENDANCE SYSTEM WITH GOOGLE SHEET”** under the guidance and supervision of **Mr. S. R. Hawal** during the academic year 2024-2025 Semester III and IV. This is a part of partial fulfillment of the requirement for award of degree of the Bachelor of Technology in Electronics and Computer Engineering.

Mr. S. R. Hawal
Mini Project Guide

Dr. P. S. Patil
HOD, ECE Department

Dr. S. A. Khot
Principal

ACKNOWLEDGEMENT

We would like to express our deep and sincere gratitude to our Guide **Mr. S. R. Hawal**, Department of Electronics and Computer Engineering, for guiding us to accomplish this project work. It was our privilege and pleasure to work under his valuable guidance. We are indeed grateful to him for providing helpful suggestion, for time to time. Due to his constant encouragement and inspiration, we are able to present this project.

We express our deep gratitude to **Dr. P. S. Patil**, Head of Electronics and Computer Engineering Department, for his valuable guidance and constant encouragement.

We are very much thankful to **Dr. S. A. Khot**, Principal, Sharad Institute of Technology College of Engineering, Yadav-Ichalkaranji, for providing all the necessary facilities to carry out project work.

Last but not least we are thankful to our parents for their moral support.

ABSTRACT

The Google Sheet Attendance Tracker using ESP8266 and a Biometric Sensor is a smart attendance management system aimed at automating the traditional attendance process. This project utilizes a fingerprint sensor for biometric authentication and an ESP8266 Wi-Fi module to connect to the internet. Once a fingerprint is recognized, the data is sent and recorded directly into a Google Sheet in real time. This system ensures accurate attendance tracking, reduces manual errors, and provides easy access to attendance records from anywhere. It is ideal for educational institutions, offices, or any organization where attendance tracking is necessary.

INDEX

1. Introduction.....	6
2. Objective.....	7
3. Problem Statement.....	8
4. Component Required.....	9
5. Flow chart & Circuit diagram.....	10
6. Working	12
7. Advantages & Dis-advantages.....	30
8. Applications	32
9. Conclusion.....	34
10. References.....	35

INTRODUCTION

In today's fast-paced world, efficient attendance management systems are crucial for educational institutions, workplaces, and other organizations. Traditional methods of manually marking attendance are not only time-consuming but also prone to errors and manipulation. To address these challenges, a Fingerprint Based Biometric Attendance System using Arduino offers a reliable and secure solution.

This project leverages the Arduino microcontroller and a fingerprint sensor module to automate the attendance process. By capturing and verifying fingerprints, the system ensures accurate and tamper-proof attendance records. The integration of an LCD display and a Real-Time Clock (RTC) module further enhances the system by providing real-time data display and timestamped records.

The primary objective of this project is to design and implement a biometric attendance system that is easy to use, cost-effective, and scalable. The system will be capable of enrolling new users, verifying fingerprints, marking attendance, and storing data for future reference. This project not only streamlines the attendance process but also adds a layer of security by using biometric data, making it a valuable tool for any organization.

OBJECTIVES

- To design and implement an attendance tracking system using biometric authentication.
- To automate the attendance process and eliminate manual errors.
- To integrate ESP8266 with Google Sheets for real-time cloud data storage.
- To provide a cost-effective and scalable solution.
- To ensure the system is user-friendly and accessible via any internet-enabled device.

PROBLEM STATEMENT

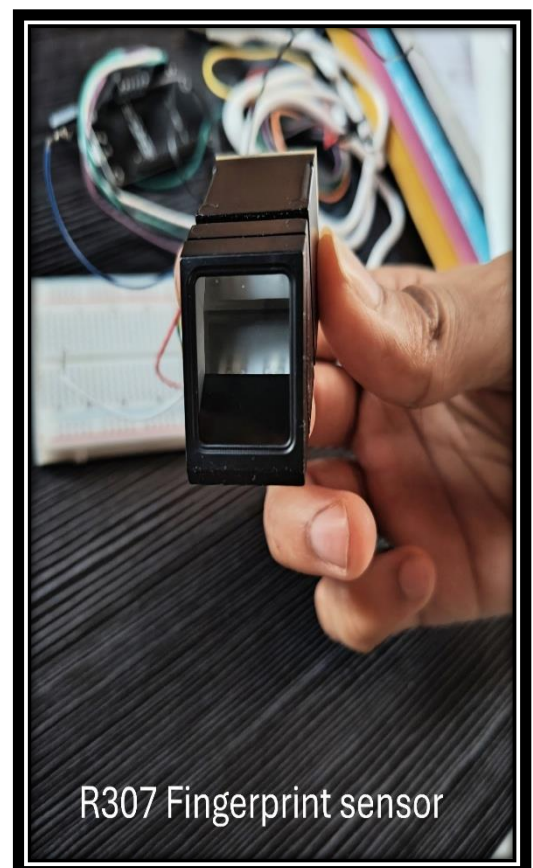
Traditional attendance systems are often inefficient, prone to proxy attendance, and involve significant administrative effort for record maintenance. There is a need for an automated, secure, and easily accessible system that can reliably track attendance and store records in real-time. The proposed solution addresses these issues by using fingerprint-based biometric authentication combined with ESP8266 to log attendance directly into Google Sheets, ensuring accuracy, security, and ease of use.

COMPONENTS RERUIRED

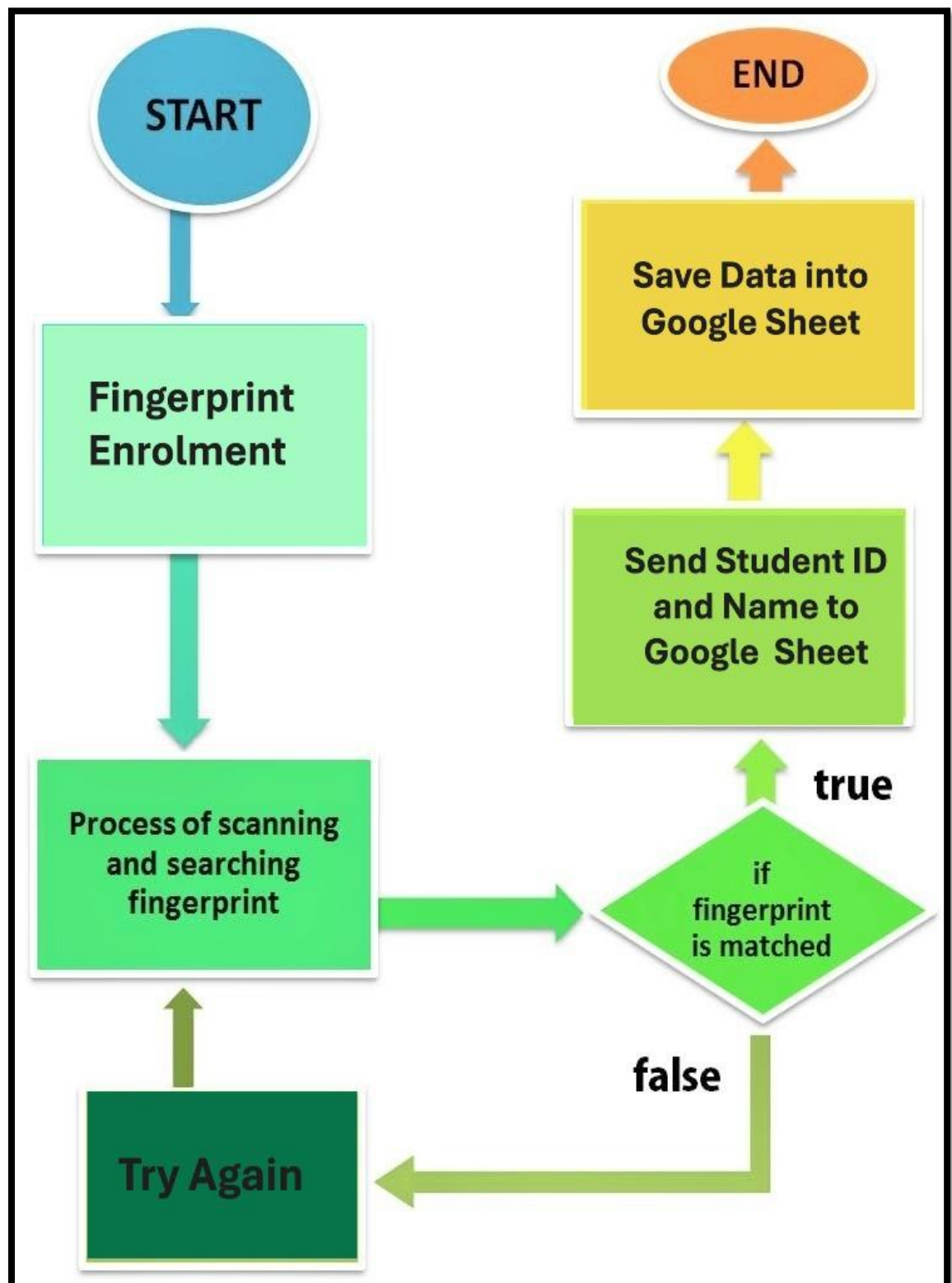
1. NodeMCU (ESP8266)
2. R307 Fingerprint sensor
3. Jumping wires
4. Power supply
5. Breadboard

R-307 Fingerprint Sensor :-

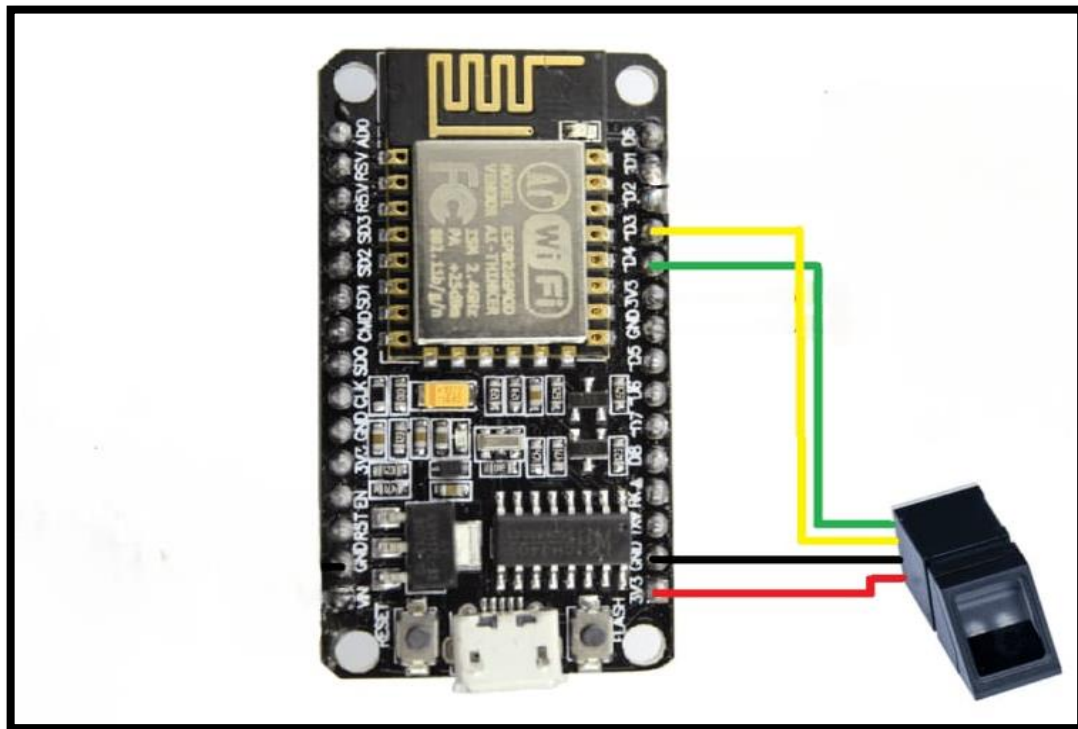
R307 Fingerprint Sensor	ESP8266 (NodeMCU)	Description
VCC (Pin 1)	Vin (5V)	Power supply (5V)
GND (Pin 2)	GND	Ground
TX (Pin 3)	D3 (GPIO0)	UART RX (Receive)
RX (Pin 4)	D4 (GPIO2)	UART TX (Transmit)
TOUCH (Pin 5)	Not Connected	Optional wake-up pin
3.3V (Pin 6)	Not Connected	Do not use



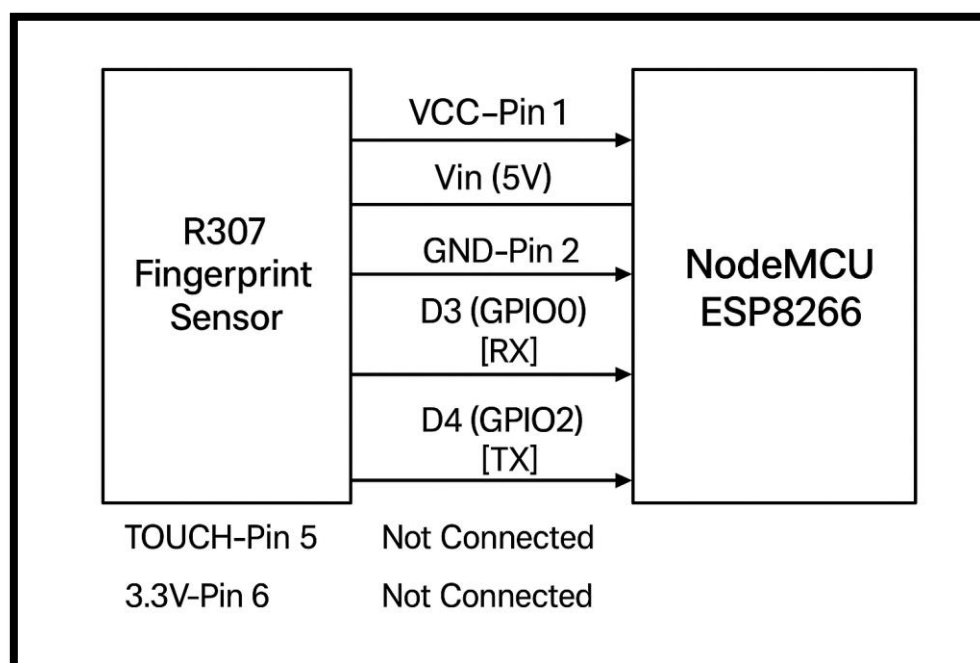
- Flow Chart :-



- **Circuit Diagram :-**



- **Block Diagram :-**



- **Working**

//Fingerprint Enrollment Process//

1. Initializing the Fingerprint Sensor

- The ESP8266 communicates with the fingerprint sensor using **UART (TX/RX pins)**.
- A library like the **Adafruit Fingerprint Sensor Library** is used to interact with the sensor.
- The system checks if the fingerprint module is properly connected.

2. Capturing a Fingerprint

- The user places a finger on the sensor.
- The sensor captures the fingerprint image and converts it into **digital data**.

3. Converting the Fingerprint to a Template

- The captured fingerprint is processed to create a **unique biometric template**.
- This template is stored in the sensor's internal memory.
- Each fingerprint is assigned a **unique ID (user ID)** for identification.

4. Confirming the Fingerprint Enrollment

- The system asks the user to place their finger again to verify the stored template.
- If the fingerprints match, enrollment is successful.
- The unique ID is stored in the ESP8266's memory for future authentication.

❑ Fingerprint Enrolment Code:

```
1#include <Adafruit_Fingerprint.h>
2#if (defined(__AVR__) || defined(ESP8266)) && !defined(__AVR_ATmega2560__)
3SoftwareSerial mySerial(D3, D4);
4#else
5#define mySerial Serial1
6#endif
7Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);
8uint8_t id;
9void setup()
10{
11  Serial.begin(9600);
12  while (!Serial); // For Yun/Leo/Micro/Zero/...
13  delay(100);
14  Serial.println("\n\nAdafruit Fingerprint sensor enrollment");
15  finger.begin(57600);
16  if (finger.verifyPassword()) {
17    Serial.println("Found fingerprint sensor!");
18  } else {
19    Serial.println("Did not find fingerprint sensor :(");
20    while (1) { delay(1); }
21  }
22  Serial.println(F("Reading sensor parameters"));
23  finger.getParameters();
24  Serial.print(F("Status: 0x")); Serial.println(finger.status_reg, HEX);
25  Serial.print(F("Sys ID: 0x")); Serial.println(finger.system_id, HEX);
26  Serial.print(F("Capacity: ")); Serial.println(finger.capacity);
27  Serial.print(F("Security level: ")); Serial.println(finger.security_level);
28  Serial.print(F("Device address: ")); Serial.println(finger.device_addr, HEX);
```

```

29 Serial.print(F("Packet len: ")); Serial.println(finger.packet_len);
30 Serial.print(F("Baud rate: ")); Serial.println(finger.baud_rate);
31}
32 uint8_t readnumber(void) {
33   uint8_t num = 0;
34   while (num == 0) {
35     while (! Serial.available());
36     num = Serial.parseInt();
37   }
38   return num;
39}
40 void loop()           // run over and over again
41{
42   Serial.println("Ready to enroll a fingerprint!");
43   Serial.println("Please type in the ID # (from 1 to 127) you want to save this finger as...");
44   id = readnumber();
45   if (id == 0) { // ID #0 not allowed, try again!
46     return;
47   }
48   Serial.print("Enrolling ID #");
49   Serial.println(id);
50
51   while (! getFingerprintEnroll() );
52}
53 uint8_t getFingerprintEnroll() {
54   int p = -1;
55   Serial.print("Waiting for valid finger to enroll as #"); Serial.println(id);
56   while (p != FINGERPRINT_OK) {
57     p = finger.getImage();
58     switch (p) {
59     case FINGERPRINT_OK:

```

```

60  Serial.println("Image taken");
61  break;
62  case FINGERPRINT_NOFINGER:
63  Serial.println(".");
64  break;
65  case FINGERPRINT_PACKETRECEIVEERR:
66  Serial.println("Communication error");
67  break;
68  case FINGERPRINT_IMAGEFAIL:
69  Serial.println("Imaging error");
70  break;
71  default:
72  Serial.println("Unknown error");
73  break;
74  }
75  }
76  p = finger.image2Tz(1);
77  switch (p) {
78  case FINGERPRINT_OK:
79  Serial.println("Image converted");
80  break;
81  case FINGERPRINT_IMAGEMESS:
82  Serial.println("Image too messy");
83  return p;
84  case FINGERPRINT_PACKETRECEIVEERR:
85  Serial.println("Communication error");
86  return p;
87  case FINGERPRINT_FEATUREFAIL:
88  Serial.println("Could not find fingerprint features");
89  return p;
90  case FINGERPRINT_INVALIDIMAGE:

```

```

91  Serial.println("Could not find fingerprint features");
92  return p;
93  default:
94  Serial.println("Unknown error");
95  return p;
96  }
97
98  Serial.println("Remove finger");
99  delay(2000);
100 p = 0;
101 while (p != FINGERPRINT_NOFINGER) {
102   p = finger.getImage();
103 }
104 Serial.print("ID "); Serial.println(id);
105 p = -1;
106 Serial.println("Place same finger again");
107 while (p != FINGERPRINT_OK) {
108   p = finger.getImage();
109   switch (p) {
110     case FINGERPRINT_OK:
111       Serial.println("Image taken");
112       break;
113     case FINGERPRINT_NOFINGER:
114       Serial.print(".");
115       break;
116     case FINGERPRINT_PACKETRECEIVEERR:
117       Serial.println("Communication error");
118       break;
119     case FINGERPRINT_IMAGEFAIL:
120       Serial.println("Imaging error");
121       break;

```



```

122  default:
123      Serial.println("Unknown error");
124      break;
125  }
126 }
127 p = finger.image2Tz(2);
128 switch (p) {
129     case FINGERPRINT_OK:
130         Serial.println("Image converted");
131         break;
132     case FINGERPRINT_IMAGEMESS:
133         Serial.println("Image too messy");
134         return p;
135     case FINGERPRINT_PACKETRECIEVEERR:
136         Serial.println("Communication error");
137         return p;
138     case FINGERPRINT_FEATUREFAIL:
139         Serial.println("Could not find fingerprint features");
140         return p;
141     case FINGERPRINT_INVALIDIMAGE:
142         Serial.println("Could not find fingerprint features");
143         return p;
144     default:
145         Serial.println("Unknown error");
146         return p;
147 }
148 Serial.print("Creating model for #"); Serial.println(id);
149 p = finger.createModel();
150 if (p == FINGERPRINT_OK) {
151     Serial.println("Prints matched!");
152 } else if (p == FINGERPRINT_PACKETRECIEVEERR) {

```

```
153 Serial.println("Communication error");
154 return p;
155 } else if (p == FINGERPRINT_ENROLLMISMATCH) {
156 Serial.println("Fingerprints did not match");
157 return p;
158 } else {
159 Serial.println("Unknown error");
160 return p;
161 }
162 Serial.print("ID "); Serial.println(id);
163 p = finger.storeModel(id);
164 if (p == FINGERPRINT_OK) {
165 Serial.println("Stored!");
166 } else if (p == FINGERPRINT_PACKETRECEIVEERR) {
167 Serial.println("Communication error");
168 return p;
169 } else if (p == FINGERPRINT_BADLOCATION) {
170 Serial.println("Could not store in that location");
171 return p;
172 } else if (p == FINGERPRINT_FLASHERR) {
173 Serial.println("Error writing to flash");
174 return p;
175 } else {
176 Serial.println("Unknown error");
177 return p;
178 }
179
180 return true;
181}
```

//Sending Attendance Data to Google Sheets//

1. Setting Up Google Sheets & Google Apps Script

- A Google Sheet is created to store attendance records (Date, Time, Name, ID, Status).

2. ESP8266 Sending Data to Google Sheets

- When a user scans their fingerprint, the system:
- Matches the fingerprint with the stored templates.
- If matched, extracts the user ID.

Sends an HTTP request to Google Sheets with the data.

3. Google Script Processing

- The script receives the request and writes the attendance data into the sheet.
- Each entry is recorded with a time, Name, ID, and status (IN /OUT).

4. Data Visualization & Access

- The Google Sheet updates in real-time.
- The data can be accessed from any device via the internet.
- You can use Google Sheets functions to analyze attendance trends.

❑ Google Sheet Attendance Tracker Code:

```
1 #include <Adafruit_GFX.h>
2 #include <Adafruit_SSD1306.h>
3 #include <Adafruit_Fingerprint.h>
4 #include <Arduino.h>
5 #include <ESP8266WiFi.h>
6 #include "HTTPSRedirect.h"
7 #include <Wire.h>
8
9 #define RELAY_PIN D2 // Define relay pin
10
11 struct User {
12     String name;
13     int id;
14     bool hasScanned; // Track if the user has scanned for IN Time
15 };
16
17 User users[] = {
18     {"Abhi", 1, false},
19     {"Aditya", 2, false},
20     {"Darshan", 3, false},
21     {"Rushi", 4, false},
22     {"Virendra", 6, false},
23     {"Anup", 7, false},
24     {"Pranav", 10, false},
25
26
27 };
28
```

```

29 bool relayState[10] = {0}; // Track relay state for each user
30
31 SoftwareSerial mySerial(D4, D3);
32 Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);
33
34 const char* ssid = "Abhi";
35 const char* password = "abhi1289";
36 const char *GScriptId = "AKfycbyiaQkKil363jgXrsMO5rRtBxm-
37 ddUPnM7OKjoes3Bm1Llhj7_m3ksvEEUsqlrapDhb";
38 const char* host = "script.google.com";
39 const int httpsPort = 443;
40 String url = String("/macros/s/") + GScriptId + "/exec";
41 HTTPSRedirect* client = nullptr;
42
43 Adafruit_SSD1306 display(128, 64, &Wire, -1);
44
45 void setup() {
46     Serial.begin(9600);
47     pinMode(RELAY_PIN, OUTPUT);
48     digitalWrite(RELAY_PIN, LOW);
49
50     WiFi.begin(ssid, password);
51     Serial.print("Connecting to WiFi...");
52     while (WiFi.status() != WL_CONNECTED) {
53         delay(1000);
54         Serial.print(".");
55     }
56     Serial.println("\nWiFi Connected!");
57

```

```

58 client = new HTTPSRedirect(httpsPort);
59 client->setInsecure();
60
61 display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
62 display.clearDisplay();
63 display.setTextColor(WHITE);
64
65 finger.begin(57600);
66 if (!finger.verifyPassword()) {
67     Serial.println("Fingerprint sensor not found!");
68     while (1) delay(1);
69 }
70 Serial.println("Fingerprint sensor found!");
71 }
72
73
74 void loop() {
75     displayMessage("Waiting...");
76     int fingerID = getFingerprintID();
77     if (fingerID > 0) {
78         processFingerprint(fingerID);
79         displayMessage("Success");
80     }
81     delay(500);
82 }
83
84 void processFingerprint(int id) {
85     for (int i = 0; i < 11; i++) {
86         if (users[i].id == id) {

```

```

87     if (!users[i].hasScanned) {
88         // First scan: IN Time
89         updatesheet(users[i].name, String(users[i].id), "IN");
90         users[i].hasScanned = true;
91     } else {
92         // Second scan: OUT Time
93         updatesheet(users[i].name, String(users[i].id), "OUT");
94         users[i].hasScanned = false; // Reset for the next scan
95     }
96     relayState[i] = !relayState[i];
97     digitalWrite(RELAY_PIN, relayState[i] ? HIGH : LOW);
98     return;
99 }
100 }
101 Serial.println("User not recognized.");
102 }
103 }
104
105 void updatesheet(String user, String id, String scanType) {
106     if (!client) {
107         client = new HTTPSRedirect(httpsPort);
108         client->setInsecure();
109     }
110     if (!client->connected()) {
111         client->connect(host, httpsPort);
112     }
113
114     String payload = "{\"command\": \"update_row\", \"sheet_name\": \"Sheet1\", \"values\": \"\" + user
115 + \",\" + id + \",\" + scanType + "\"}";
116     Serial.println("Publishing data...");

```

```
116 Serial.println(payload);
117 if (!client->POST(url, host, payload)) {
118     Serial.println("Error while connecting to Google Sheets");
119 }
120 delay(5000);
121 }
122
123 void displayMessage(String msg) {
124     display.clearDisplay();
125     display.setTextSize(2);
126     display.setCursor(0, 10);
127     display.print(msg);
128     display.display();
129 }
130
131 int getFingerprintID() {
132     if (finger.getImage() != FINGERPRINT_OK) return -1;
133     if (finger.image2Tz() != FINGERPRINT_OK) return -1;
134     if (finger.fingerSearch() != FINGERPRINT_OK) return -1;
135     Serial.print("Found ID #"); Serial.println(finger.fingerID);
136     return finger.fingerID;
137 }
138 }
```


❏ Google Sheet App Script Code:

```
1 var SS = SpreadsheetApp.openById('1ErP2qHZLXpFs19Om0RhyUra7aVEeP-3KDcdIrby2_Ks');
2 var str = "";
3
4 function doPost(e) {
5     var parsedData;
6     var result = {};
7     try {
8         parsedData = JSON.parse(e.postData.contents);
9     } catch(f) {
10         return ContentService.createTextOutput("Error in parsing request body: " + f.message);
11     }
12
13     if (parsedData !== undefined) {
14         var sheet = SS.getSheetByName(parsedData.sheet_name);
15         var dataArr = parsedData.values.split(",");
16         var date_now = Utilities.formatDate(new Date(), "IST", "yyyy/MM/dd");
17         var time_now = Utilities.formatDate(new Date(), "IST", "HH:mm:ss"); // Ensure 24-hour format
18         var user = dataArr[0];
19         var id = dataArr[1];
20         var scanType = dataArr[2];
21
22
23         // Determine the subject based on the current day and time
24         var temp = determineSubject(time_now); // Use helper function to determine subject
25
26         if (scanType === "IN") {
27             // For IN Time, always append a new row
28
```

```

29     sheet.appendRow([date_now, user, id, temp, time_now, ""]);
30 } else if (scanType === "OUT") {
31     // For OUT Time, find the most recent row for the user and update the OUT Time
32     var data = sheet.getDataRange().getValues();
33     for (var i = data.length - 1; i >= 1; i--) {
34         if (data[i][1] == user && data[i][2] == id && data[i][5] == "") {
35             sheet.getRange(i + 1, 6).setValue(time_now); // Update OUT Time in column F
36             break;
37         }
38     }
39 }
40
41 str = "Success";
42 SpreadsheetApp.flush();
43 return ContentService.createTextOutput(str);
44 } else {
45     return ContentService.createTextOutput("Error! Request body empty or in incorrect format.");
46 }
47 }
48
49
50 // Helper function to determine the subject based on day and time
51 function determineSubject(time_now) {
52     var time = time_now.split(":").join(""); // Remove colons for easier comparison
53     var day = new Date().getDay(); // 0 for Sunday, 1 for Monday, ..., 6 for Saturday
54
55     if (day === 0 || day === 7) { // Sunday or Saturday
56         return "Unknown"; // No classes on weekends
57     }

```

```

58
59 if (time >= "0930" && time < "1030") {
60     if (day === 1) return "OS"; // Monday
61     if (day === 2) return "OE"; // Tuesday
62     if (day === 3) return "MDM"; // Wednesday
63     if (day === 4) return "ADC"; // Thursday
64     if (day === 5) return "MIL"; // Friday
65     if (day === 6) return "MDM"; //Sat
66 } else if (time >= "1030" && time < "1130") {
67     if (day === 1) return "MDM"; // Monday
68     if (day === 2) return "OS"; // Tuesday
69     if (day === 3) return "ADC"; // Wednesday
70     if (day === 4) return "DEM"; // Thursday
71     if (day === 5) return "ADC"; // Friday
72     if (day === 6) return "DEM"; //Sat
73 } else if (time >= "1130" && time < "1210") {
74     return "Lunch Break";
75 } else if (time >= "1210" && time < "1310") {
76     if (day === 1) return "ADC"; // Monday
77     if (day === 2) return "LAB"; // Tuesday
78     if (day === 3) return "MINIPROJECT"; // Wednesday
79     if (day === 4) return "Activity Slot"; // Thursday
80     if (day === 5) return "Language Skill-II"; // Friday
81     if (day === 6) return "APT"; //Sat
82 } else if (time >= "1310" && time < "1410") {
83     if (day === 1) return "DEM"; // Monday
84     if (day === 2) return "LAB"; // Tuesday
85     if (day === 3) return "MINIPROJECT"; // Wednesday

```

```

87     if (day === 4) return "Activity Slot";
88     if (day === 5) return "Language Skill-II";
89     if (day === 6) return "MIL"; //Sat
90 } else if (time >= "1410" && time < "1420") {
91     return "Short Break";
92 } else if (time >= "1420" && time < "1520") {
93     if (day === 1) return "APT"; // Monday
94     if (day === 2) return "OE"; // Tuesday
95     if (day === 3) return "LAB"; // Wednesday
96     if (day === 4) return "LAB"; // Thursday
97     if (day === 5) return "DEM"; // Friday
98     if (day === 6) return "OS"; //Sat
99 } else if (time >= "1520" && time < "1620") {
100     if (day === 1) return "Math"; // Monday
101     if (day === 2) return "MDM"; // Tuesday
102     if (day === 3) return "LAB"; // Wednesday
103     if (day === 4) return "LAB"; // Thursday
104     if (day === 5) return "OS"; // Friday
105     if (day === 6) return "OE"; //Sat
106
107
108 } else {
109     return "Unknown"; // Default value for times outside defined ranges
110 }
    }

```

• **Advantages:**

1. Accuracy:

- Eliminates the possibility of proxy attendance, ensuring accurate and reliable attendance records.

2. Security:

- Fingerprints are unique to each individual, making it a secure method of verifying identity.

3. Convenience:

- Users do not need to remember cards or passwords, simplifying the attendance process.

4. Efficiency:

- Speeds up the attendance-taking process, reducing time spent on roll calls or manual entries.

5. Cost-Effective:

- Arduino-based systems are relatively inexpensive compared to commercial biometric systems.

6. Easy Integration:

- Can be easily integrated with existing systems and expanded with additional features like data storage on an SD card.

7. Environmentally Friendly:

- Reduces the need for paper-based attendance registers, promoting a more sustainable approach.

- **Disadvantages :**

1. Initial Setup:

- Requires initial investment in hardware and time for setting up the system and enrolling users.

2. Maintenance:

- Regular maintenance is needed to ensure the fingerprint sensor remains clean and functional.

3. Privacy Concerns:

- Collecting biometric data raises privacy issues; users need to trust that their data is stored securely.

4. Environmental Factors:

- Fingerprint sensors can sometimes struggle with dirty, wet, or damaged fingerprints, leading to false rejections or errors in attendance.

• **Application :**

1. Educational Institutions:

- **Schools and Colleges:** Automate the attendance process for students and staff, reducing the time spent on roll calls and minimizing absenteeism.
- **Examination Centers :** Securely verify the identity of students appearing for exams, preventing impersonation.

2. Corporate and Business Environments:

- **Office Attendance:** Streamline the process of employee check-ins and check-outs, ensuring accurate attendance tracking.
- **Remote Work Monitoring:** Verify remote or field employees' attendance through portable biometric devices.

3. Government and Public Sector:

- **Municipal Offices:** Improve efficiency in local government offices by automating staff attendance.
- **Public Services:** Enhance security and accountability in public service departments, such as police and fire stations.

4. Healthcare Facilities:

- **Hospitals and Clinics:** Track attendance of medical staff and ensure timely availability of healthcare professionals.
- **Nursing Homes:** Monitor attendance of caregivers to ensure consistent and reliable care for residents.

• Outcome :

	A	B	C	D	E	F	G	H
	Table1							
1	Date	Name	# ID	Subject	IN Time	OUT Time	Total Time	Presenty
63	2025/03/18	Abhi	1	LAB	12:19:26	12:19:39	00:00:13	☐
64	2025/03/18	Anup	7	LAB	12:19:55	12:20:25	00:00:30	☒
65	2025/03/18	Abhi	1	LAB	12:20:54	12:22:09	00:01:15	☒
66	2025/03/18	Darshan	3	LAB	12:21:18	12:21:40	00:00:22	☒
67	2025/03/18	Virendra	7	LAB	12:22:20	12:22:40	00:00:20	☒
68	2025/03/18	Abhi	1	LAB	12:23:29	12:23:40	00:00:11	☐
69	2025/03/18	Aditya	2	LAB	12:24:50	12:25:12	00:00:22	☒
70	2025/03/18	Darshan	3	LAB	12:25:34	12:25:59	00:00:25	☒
71	2025/03/18	Pranav	10	LAB	12:27:37	12:28:12	00:00:35	☒
72	2025/03/18	Aditya	2	LAB	12:29:07	12:29:40	00:00:33	☒
73	2025/03/18	Abhi	1	Unknown	18:42:51	18:43:32	00:00:41	☒
74	2025/03/18	Aditya	2	Unknown	18:49:37	18:49:53	00:00:16	☐
75	2025/03/18	Darshan	3	Unknown	18:50:11	18:52:08	00:01:57	☒
76	2025/03/26	Rushi	4	Unknown	19:09:48	19:11:31	00:01:43	☒
77	2025/04/02	Virendra	6	MINIPROJECT	13:07:39	13:09:55	00:02:16	☒
78	2025/04/02	Anup	7	MINIPROJECT)	13:19:27	13:19:57	00:00:30	☒
79	2025/04/02	Pranav	10	MINIPROJECT)	13:19:43	13:20:23	00:00:40	☒

• **Conclusion :**

The Fingerprint Based Biometric Attendance System using Arduino is an effective and reliable way to manage attendance. It ensures accuracy, prevents fraud, and simplifies the attendance process. This system highlights the integration of modern technology for daily administrative tasks, showcasing the potential of biometric solutions to revolutionize traditional methods. While initial setup and maintenance are necessary, the benefits of enhanced security, efficiency, and user-friendliness make it a valuable tool for various applications.

• **Future Scope :**

1. **Cloud Integration:**

The system can be expanded to include real-time cloud storage and monitoring by integrating platforms like Firebase, AWS IoT, or Google Sheets for remote access and analytics.

2. **Mobile App Development:**

A dedicated mobile application can be developed for administrators to view, manage, and receive alerts on attendance or access logs.

3. **Multi-factor Authentication:**

Enhance security by combining fingerprint authentication with RFID, facial recognition (using ESP32-CAM), or OTP-based login systems.

- **Reference :**

- ❑ **"Biometric Attendance Systems: A Comprehensive Guide"** by Truein:

This book provides an in-depth look at how biometric attendance systems work, their benefits, and how to choose the right system for your needs.

- ❑ **"Biometric Based Attendance System"** by Kar Nirmalya, Saha Ashim, and Debbarma Mrinal Kanti:

This book covers the implementation of biometric attendance systems, including face recognition technology and PCA algorithms.